CS353: Computer Networks — Lab 10 / Lab 11 Submission

Topic: Inter-Domain Routing with Border Gateway Protocol (BGP)
Student Name: Sayan
Department: Computer Science & Engineering, IIIT Guwahati

---

1. Simulation Results (Scenarios 1, 2, and 3)

The Python implementation (`bgp_simulator.py`) implements the BGP decision process, loop prevention, and synchronous convergence.

Scenario 1: Default Policy (Local Preference = 100 for all ASes)

```
============================================================
Scenario 1: Converged Routing Table (Default Policy)
============================================================
AS     AS-path                Next hop
------------------------------------------------------------
AS2    AS1                    AS1
AS3    AS1                    AS1
AS4    AS2, AS1               AS2
AS5    AS3, AS1               AS3
AS6    AS4, AS2, AS1          AS4
============================================================
```

Verification: AS6 receives two equal-length paths of length 3 (`[4, 2, 1]` via AS4 and `[5, 3, 1]` via AS5). The tie-break rule (Rule 3: lowest next-hop AS) chooses AS4 ($4 < 5$).

Scenario 2: AS6 sets Local Preference 200 on routes from AS5

```
============================================================
Scenario 2: Converged Routing Table (AS6 prefers AS5 with LocPrf
============================================================
AS     AS-path                Next hop
```

```
AS     AS-path                  Next hop
------------------------------------------------------------
AS2    AS1                      AS1
AS3    AS1                      AS1
AS4    AS2, AS1                 AS2
AS5    AS3, AS1                 AS3
AS6    AS5, AS3, AS1            AS5
============================================================
```

Scenario 3: AS5 sets Local Preference 50 on routes from AS3

```
============================================================
Scenario 3: Converged Routing Table (AS5 de-prefers AS3 with Loc
============================================================
AS     AS-path                  Next hop
------------------------------------------------------------
AS2    AS1                      AS1
AS3    AS1                      AS1
AS4    AS2, AS1                 AS2
AS5    AS4, AS2, AS1            AS4
AS6    AS4, AS2, AS1            AS4
============================================================
```

Verification: AS5 deliberately chooses the longer length-3 path via AS4 (`[4, 2, 1]`) instead of the shorter length-2 path via AS3 (`[3, 1]`) because the default local preference of 100 on AS4 exceeds the configured local preference of 50 on AS3.

---

2. Part D: Written Answers to Conceptual Questions

Question 1: Local Preference Override in Scenario 2

> **Prompt:** Apply Scenario 2 (AS6 prefers routes from AS5) and confirm only AS6 changes, to the route via AS5. Explain why local preference overrides the AS-path tie-break.

Answer:
In the BGP decision process, **Rule 1 (Highest Local Preference)** is evaluated strictly prior to Rule 2 (Shortest AS-Path Length) and Rule 3 (Lowest Next-Hop AS Number). In Scenario 2, AS6 receives two candidate routes to prefix `203.0.113.0/24`:

1. Via AS4: AS-path `[AS4, AS2, AS1]`, length 3, default Local Preference = 100, next-hop = AS4.
2. Via AS5: AS-path `[AS5, AS3, AS1]`, length 3, configured Local Preference = 200, next-hop = AS5.

Because $200 > 100$, candidate route via AS5 wins decisively at Rule 1. The decision process terminates immediately without ever proceeding to the next-hop tie-break (which favored AS4 in Scenario 1 because $4 < 5$). Only AS6 changes its routing decision because AS6 is a stub/leaf node in this topology; no other AS relies on AS6 to reach AS1.

Question 2: Deliberate Choice of a Longer Path & Dijkstra Comparison

> **Prompt:** Apply Scenario 3 (AS5 de-prefers routes from AS3) and confirm AS5 installs the length-3 path via AS4 instead of the length-2 path via AS3. State plainly that AS5 has chosen a longer path, and explain why this cannot happen in the Dijkstra sink-tree lab but is normal in BGP.

Answer:
Plain Statement: AS5 has deliberately chosen a longer path: it installs the length-3 path `[AS4, AS2, AS1]` via next-hop AS4 instead of the shorter length-2 path `[AS3, AS1]` via next-hop AS3.

Why this cannot happen in the Dijkstra sink-tree lab:
In Dijkstra's algorithm (used in intra-domain link-state routing protocols like OSPF), all routers share a globally synchronized, identical link-state database (LSDB) and optimize for a single, globally objective metric—minimizing total path cost. Dijkstra's algorithm mathematically constructs a global shortest-path tree (sink tree) where every sub-path must be strictly optimal with respect to metric addition. A router running Dijkstra cannot independently choose an inferior, longer path based on subjective local preference because metric optimization is global, uniform, and additive.

Why this is normal in BGP:
BGP is an inter-domain routing protocol governing independent, sovereign

BGP is an inter-domain routing protocol governing independent, sovereign administrative domains with distinct commercial and contractual relationships (customer-provider, peer-peer). Inter-domain routing is fundamentally driven by business policy rather than geometric distance. For example, if AS3 is an expensive upstream provider that charges for transit, while AS4 is a settlement-free peer or paying customer, AS5 lowers the local preference of routes through AS3 to avoid financial cost. Prioritizing business policy over shortest hop count is standard operational practice in the global Internet.

Question 3: Loop Prevention via AS-Path vs. Count-to-Infinity

> **Prompt:** Explain how the AS-path prevents routing loops, and why this means BGP does not suffer the count-to-infinity problem that a distance-vector protocol would on the same topology.

Answer:
Loop Prevention:
BGP is a path-vector protocol. Whenever an AS advertises a route to an external BGP neighbor, it prepends its own Autonomous System Number (ASN) to the `AS_PATH` attribute. When an AS receives a candidate route advertisement, the very first step before route selection is the **loop check**: the router scans the `AS_PATH` list. If its own ASN is already present in the list, the candidate route is recognized as a cyclic loop and is immediately discarded.

Elimination of Count-to-Infinity:
In traditional distance-vector protocols (such as RIP), routers exchange only a scalar cost metric (hop count) without path history. When a link fails, mutual dependency loops form (A routes through B, while B routes through A), and routers repeatedly increment the metric by 1 until reaching an artificial infinity threshold.
In contrast, because BGP carries the complete history of traversed ASes in the `AS_PATH`, any route propagating back to an AS that previously handled it is identified and dropped instantly in a single iteration. Routing loops are eliminated deterministically, so BGP never suffers from the count-to-infinity problem.

Question 4: Necessity of Policy-Based Path Selection

> **Prompt:** In two or three sentences, explain why inter-domain routing uses policy-based path selection at all, rather than simply minimising the number of AS hops.

Answer:
Inter-domain routing interconnects sovereign, competing commercial and national networks that have distinct business relationships, economic incentives, and legal or security constraints. Simply minimising AS hops would force networks to carry expensive or uncompensated transit traffic for competitors, violating financial agreements and transit policies. Policy-based path selection (e.g., via Local Preference) enables each Autonomous System to prioritize commercially profitable paths, honor customer agreements, and retain administrative control over its traffic.

---

3. Part E: Real BGP Capture Analysis (`lab11_bgp_session.cap`)

1. **Protocol & Message Types:**
   - Transport: **TCP Port 179**.
   - Identified Message Types: **`OPEN`** (Frames 4 & 5), **`KEEPALIVE`** (Frames 6 & 7), and **`UPDATE`** (Frame 8).
2. **UPDATE Message Path Attributes (Frame 8):**
   - **ORIGIN:** IGP (`0x00`)
   - **AS_PATH:** Sequence `4, 2, 1` (Segment type `AS_SEQUENCE`, length 3 ASNs: 4, 2, 1)
   - **NEXT_HOP:** `198.51.100.4`
   - **NLRI (Advertised Prefix):** `203.0.113.0/24`
3. **One-Sentence Reconciliation Statement:**

> *"The ordered sequence `4, 2, 1` in the `AS_PATH` segment of the BGP UPDATE packet corresponds directly to the Python simulator list `[4, 2, 1]`, representing the exact AS trajectory (via next-hop AS4, through transit AS2, to origin AS1) installed by AS6 in Scenario 1."*

---

4. Part F: Real Routers with FRRouting (Docker Containers)

Topology:

`r1 (AS65001)` → `r2 (AS65002)` → `r3 (AS65003)`
Where `r1` originates `203.0.113.0/24`.

Output 1: Before Policy Change (On `r3`)

```
r3# show ip bgp 203.0.113.0/24
BGP routing table entry for 203.0.113.0/24, version 1
Paths: (1 available, best #1, table default)
  Advertised to non peer-group peers:
    10.0.23.2
  65002 65001
    10.0.23.2 from 10.0.23.2 (10.0.12.2)
      Origin IGP, valid, external, best (First path received)
      Last update: Thu Nov  5 17:09:07 2026
```

Result: The AS-path is `65002 65001`, confirming transit AS `r2` prepended its own ASN (`65002`) to the path received from `r1` (`65001`).

Applied Policy Change: AS-Path Prepending on `r2`

```
docker exec -it r2 vtysh -c "configure terminal" \
  -c "route-map PREPEND permit 10" \
  -c "set as-path prepend 65002" \
  -c "router bgp 65002" \
  -c "neighbor 10.0.23.3 route-map PREPEND out" \
  -c "end" \
  -c "clear ip bgp 10.0.23.3 soft out"
```

Output 2: After Policy Change (On `r3`)

```
r3# show ip bgp 203.0.113.0/24
BGP routing table entry for 203.0.113.0/24, version 2
Paths: (1 available, best #1, table default)
```

```
    Advertised to non peer-group peers:
      10.0.23.2
    65002 65002 65001
      10.0.23.2 from 10.0.23.2 (10.0.12.2)
        Origin IGP, valid, external, best (First path received)
        Last update: Thu Nov  5 17:13:50 2026
```

Result: The AS-path has grown to **`65002 65002 65001`** (length 3).

Packet Capture:

The live BGP session was captured on the `r2` - `r3` link and saved as: `captured_bgp_update.pcap` Wireshark inspection confirms the BGP UPDATE message carries `AS_PATH: 65002 65002 65001` and prefix `203.0.113.0/24`.